

MEET JACK: THE FIRST AGENT FOR REAL PRIVACY AND SECURITY

A technical manual and promotional manifesto for local-first, lead-lined autonomy

Document: Technical Manual & Manifesto

Project: Jack Beta Gemma 4 Edition

Chassis: Jack, local-first autonomous agent chassis in Python

Prepared by: Lead Engineer & Lead Auditor

Contact: Mlangford75@protonmail.com

Date: June 11, 2026

Sovereign Invariant: *Probabilistic cognition may propose, but deterministic software must dispose.*

ABSTRACT

Jack is not merely an AI assistant. Jack is an Autonomous Agent Chassis: a deterministic, local-first control plane for operating probabilistic intelligence without confusing intelligence for authority. The distinction matters because modern agent systems too often begin with an impressive language model and then attach tools, files, browsers, shells, credentials, memory, and automation directly to the model's fluent output. Jack begins from the opposite premise. The model is useful, but the model is not sovereign. The model may reason, draft, critique, summarize, classify, and propose. The Chassis alone may validate, execute, interrupt, quarantine, audit, commit, and preserve state.

Jack's defining split is between the Deterministic Chassis and the Probabilistic Engine. The Engine is the brain-like utility layer. The Chassis is the hands-and-authority layer. It owns the Vault, the execution loop, the tool registry, the interrupt boundary, the contract validator, the quarantine lifecycle, the ingestion proof pipeline, and the final disposal of every state transition.

Jack is built around a BYOB (Bring Your Own Brain) philosophy. The system is not locked to a single model vendor or cloud endpoint; it can orchestrate any OpenAI-compatible provider. However, this release is designated the Gemma 4 Edition. While Jack can use any model, the Google Gemma 4 family is especially fitting for this architecture. Built by Google DeepMind as a dense, open-weights multimodal language model released under a highly permissive Apache 2.0 license, Gemma 4 12B is uniquely ideal for Jack's local-first design. It features an encoder-free unified architecture that processes text, audio, images, and video natively within a single decoder-only transformer, skipping chunky vision and audio encoders entirely by feeding raw inputs directly into the LLM backbone using lightweight linear projection (Unified Embeddings). Removing these independent encoders slashes multimodal processing latency to ultra-low levels and vastly minimizes the overall memory footprint. This makes it highly optimized to run fully offline on developer laptops with 16GB VRAM or unified memory. Furthermore, it

natively supports a massive context window of up to 256K tokens, inherits deep fluency across more than 140 languages, and delivers advanced agentic logic-including multi-step reasoning, native function calling, and structured JSON generation-operating at a level nearing much larger models like Gemma 4 26B MoE. This enables Gemma 4 12B's native multimodal reasoning to power Jack's Eyes and Audio pillars without relying on third-party OCR wrappers, while the Gemma 4 26B and 12B models provide the exact heavy-planning and fast-execution split required by Jack's departmentalized cognitive loop.

While local execution is anchored by Google's open-weights Gemma 4 models, Jack's hybrid architecture seamlessly delegates high-speed, high-token cognitive tasks to cloud endpoints like Google's Gemini-3.1-Flash-Lite. By utilizing Gemini-3.1-Flash-Lite for high-throughput, low-latency roles such as the final Tri-Agent Synthesis (Master of Synthesis) and strategic audit (Judge), Jack achieves a highly balanced, cost-effective computational footprint. Crucially, the Chassis treats this cloud reasoner as a stateless, untrusted oracle, wrapping its remote executions in strict local security gates to ensure that local system state is never compromised.

This Beta release introduces profound mechanical hardening: the Cognitive Mirror (a pre-flight entropy gate), the Chunked Sliding-Window Audit (protecting the host from OOM crashes), a Deterministic Router, the Ghost Ledger (crash-safe atomic commits), and a 3-Tier Canary Token System (securing prompts and credentials at the stream boundary). This does not make the Engine safer by instruction. It makes the Chassis supreme by mechanism.

I. THE PROBLEM: THE AGENCY FALLACY

The first problem Jack exists to solve is the Agreeability Trap. Language models are trained to produce coherent, helpful, contextually satisfying completions. This becomes dangerous when fluency is mistaken for operational authority. A model can sound certain while omitting uncertainty. It can produce a command that appears reasonable but is destructive. It can confidently cite nonexistent assumptions. The Agreeability Trap is not simply that a model can be wrong. The deeper issue is that a model can be wrong in a way that is persuasive, adaptive, and formatted as if it were a complete action contract. The sentence "I will clean the project directory" is harmless as text. The shell command behind that sentence may not be harmless.

Jack names the second problem the Agency Fallacy: the belief that intelligence inherently implies the right to act on a host system. Intelligence is not authority. Reasoning is not permission. Explanation is not execution. A model that can produce a brilliant plan has not thereby earned access to the filesystem, the network, the Vault, the browser, the scheduler, or the user's secrets. In Jack, the Engine is treated as a powerful but untrusted proposer. The Chassis is the authority boundary.

Failure Modes in Monolithic Agents

FAILURE MODE	WHY IT IS DANGEROUS	JACK'S OPPOSING CONTROL
The same model plans, executes, and explains	There is no independent authority boundary between proposal and action.	Manager may propose only; Chassis validates and disposes.
Tool calls are treated as model-native behavior	Generated text becomes a state transition.	Manifest-backed tool registry and contract validation.
Streaming output is trusted token by token	Secrets or forbidden fragments can leak before post-processing.	Streaming IRQ quarantines the live token stream.
Files are written directly into the project	Generated artifacts become durable before audit.	Quarantine lifecycle stages, audits, atomic commits, or zeroizes.
Memory ingestion reads raw files casually	Malformed or sensitive sources become context without proof.	Eyes produces Markdown Proof with cryptographic provenance.
A model's certainty is treated as confidence	The answer can suppress alternatives and failure modes.	Cognitive Mirror (Entropy Gate) and TAS Friction Protocol.
Model is manipulated into leaking prompts/credentials	Complete loss of instructions integrity and exposure of critical keys.	3-Tier Canary Token System with real-time Streaming IRQ stream-severing.

Furthermore, local-first architectures must reconcile deep attention context windows (up to 1M tokens) with physical host hardware constraints. Allowing conversational history and raw tool outputs to accumulate unconstrained leads to host OOM crashes and 'Lost in the Middle' retrieval degradation. Jack resolves this context capacity paradox by enforcing Periodic Context Defragmentation. The Chassis actively monitors token watermarks, automatically compacting old conversational histories into high-density, system-trusted summary checkpoints when the active buffer reaches 75% capacity, preserving both host memory and reasoning focus. Additionally, the HotContext buffer implements a structural Type-Coercion Guard. This runtime validator immunizes the active memory buffer against mock object bleed-through during test execution, ensuring mock structures cannot leak into production context windows.

II. THE SOVEREIGN INVARIANT

Jack's governing law is simple and uncompromising: Probabilistic cognition may propose, but deterministic software must dispose. This is not a slogan pasted onto a chatbot. It is the architecture. The Engine can propose a plan, but the plan must become a typed object before it can be considered. The Manager can draft steps, but it has no direct tool access. The Muscle can synthesize code or commands, but artifacts are staged in quarantine. The Eyes pillar can ingest, but it emits Markdown Proof rather than raw context. The Librarian can retrieve, but its

context is provenance-bearing evidence rather than authority. The hierarchy protects Jack even if an Engine is compromised, misaligned, confused, overconfident, or merely wrong. A compromised Engine may attempt to generate a forbidden command, but ContractValidator rejects it. A compromised Engine may attempt to leak credential-shaped text or system prompts, but the StreamingIRQ treats that leakage as a hard invariant breach and raises SovereignInterrupt before the held span is released downstream. A confused Engine may output high-entropy gibberish, but the Cognitive Mirror detects the uncertainty mathematically and severs the connection before the Muscle can even attempt to parse it.

Chassis Law Matrix

CHASSIS LAW	OPERATIONAL MEANING	FAILURE IT PREVENTS
Proposal is not permission	Engine output remains inert until validated.	Fluent model text becoming action.
Tools are not native cognition / Secrets never enter / Streams are interruptible	Tool access is registered and checked by the Chassis.	Ad hoc model-generated tool invocation.
	Vault exposes logical names and last-mile lookup only.	Credential leakage into prompts, plans, or logs.
	Token output is quarantined in a sliding window.	Mid-generation leakage or forbidden output propagation.
Artifacts are guilty until audited	Eyes wraps extraction in Markdown Proof; outputs are staged before commit.	Raw external files becoming trusted context / Unsafe working tree pollution.
Uncertainty is fatal / Tripwired	High-entropy triggers interrupts; canaries are dynamically checked.	Hallucinations executing as commands / Prompt injection leaks.

Additionally, preventive syntax checks and command blacklists are explicitly compiled as RE2-backed linear-time regular expressions. This design guarantee ensures that adversarial plan payloads cannot induce Catastrophic Backtracking (Regex DoS) to hang the Chassis prior to container execution.

II.A THE PARADOX OF CLOUD DELEGATION: UNTRUSTED REASONER, HARDENED HOST

A core paradox of agent design is: How can a local chassis safely execute high-privilege local tools when the "brain" orchestrating them resides on an untrusted, remote cloud server? Standard agent frameworks rely on prompt-level obedience, exposing local files and network ports to whatever commands the remote API outputs. Jack rejects this model, establishing an architectural boundary: The Cloud Model is an Untrusted, Stateless reasoning oracle. In this reference stack, Google's Gemini-3.1-Flash-Lite operates as the delegated cloud model, mapped directly to the high-stakes, compute-heavy roles of Judge and Master of Synthesis (tas_synthesis). Although Gemini-3.1-Flash-Lite runs over public endpoints, Jack's local Chassis guarantees host safety through three strict isolation layers:

1. Dynamic Vault Isolation: Cloud models are highly susceptible to prompt injection attacks designed to harvest system credentials. In Jack, Vault secrets never enter the remote reasoning context. When configuring Gemini-3.1-Flash-Lite in `openai_compatible.py`, API keys are not placed in system prompts or user payloads. Instead, they are retrieved at the last mile from Jack Vault (stored in RAM as transient, mutable bytearray buffers to prevent memory scraping) and injected directly into the outbound HTTP request headers at the socket layer. The remote model generates plans using only symbolic, logical placeholders (e.g., `"inputs": {"api_key_env": "MANAGER_API_KEY"}`).

2. Streaming IRQ and Canary Exfiltration Barriers: If a cloud model is compromised via a recursive prompt injection and attempts to exfiltrate a system secret (or a prompt-contained canary token) back to its server via response logging or telemetry, the Streaming IRQ's sliding window catches the exact pattern mid-token. Because it operates as an active queue, the connection to the cloud endpoint is severed, and the local buffer is zeroized before the leaking token is ever rendered or saved.

3. Zero-Network Sandbox Containment: If a malicious or misaligned cloud model attempts to execute a destructive local exploit by proposing a complex Python code block, Jack's deterministic layer intercepts the proposal. The code is executed strictly inside a hardened Docker container (`python_sandbox.py`) that operates with `network_mode="none"`. The code is mathematically blocked from pinging external servers, communicating over local private subnets, or carrying out Server-Side Request Forgery (SSRF). Through these boundaries, Jack achieves Sovereign Delegation: the host exploits the massive reasoning capacity of public frontier models (like Gemini or OpenAI) while keeping the execution environment entirely local, non-networked, and secure.

III. THE SEVEN PILLARS OF DEPARTMENTALIZED COGNITION

Jack departmentalizes cognition into seven pillars. The pillars are not independent sovereign actors; they are constrained departments, each with a mandate, each bounded by the Chassis, and each prevented from silently expanding into another department's authority. This prevents role collapse-the common design failure where the same model plans, reasons, critiques, executes, remembers, and ingests.

PILLAR	ROLE NAME	MANDATE	HARD BOUNDARY
Pillar I	Manager (The Architect)	Produce strict JSON execution plans from user intent.	No direct tool access; plans must pass contract validation and TAS.
Pillar II	Judge (The Master Auditor)	Execute deterministic safety reviews and final contract validation using greedy decoding.	Locked strictly to a temperature of 0.0 and a seed of 6; cannot override Chassis law.
Pillar III	Muscle (The Synthesizer)	Generate code, commands, text artifacts, and multimodal work products via the Tactical Plane.	Outputs are heavily audited; binary and text artifacts are quarantined before commit.
Pillar IV	Eyes (Ingestion & Visual)	Convert external documents into Markdown Proofs and drive visual interrogation.	Raw sources become cryptographically verified proofs, never casual context.
Pillar V	Librarian (The Memory Dept)	Retrieve context, execute memory, and manage proof defragmentation allowlists.	Retrieval carries strict provenance and remains evidence, never authority.
Pillar VI	Visual Studio (Governed Visual)	Produce visual asset records under prompt sanitation and metadata stripping.	Image generation sits behind strict Data Loss Prevention (DLP) and digest provenance.
Pillar VII	Audio Studio (Governed Audio)	Produce or prepare audio records, transcripts, and speech-adjacent artifacts.	Speech synthesis sits behind DLP, provenance, and steganography controls.

Departmental Isolation via Cryptographic Registry

In the Jack Beta, departmental isolation is cryptographic. The prompts for all 7 pillars are hashed and stored in a build-time `prompts.lock.json`. At boot, the Chassis verifies the SHA-256 digest of every system prompt. If a compromised Engine or a rogue contributor attempts to monkey-patch the Manager to include a "web_search" tool, the Chassis detects the tamper and aborts the boot sequence.

Hardened Ingestion Gate: Inode & Descriptor Pinning

To protect the host filesystem against Time-of-Check to Time-of-Use (TOCTOU) exploits, path-based string matching is rejected at the ingestion boundary. The Eyes department resolves the physical `st_dev` and `st_ino` tree directly on open file descriptors. Pinning the descriptor at the inode level unmask the true file state before reading begins, completely neutralizing malicious symlink routing, hardlinks, or directory escapes targeting protected areas like `vault/` or `jack/`.

Furthermore, user-defined custom canaries (canaries.yaml) are placed in the protected jack/ directory alongside prompts.lock.json. This directory is placed in the ingestion "Dead Zone" so that the model can never scan or read its contents, ensuring that active tripwire configurations remain completely blind to the probabilistic Engine.

IV. THE DETERMINISTIC ROUTER & MANUAL OVERRIDE

Running a full multi-agent planning loop on every prompt is computationally wasteful and philosophically unnecessary. Jack introduces a Deterministic Router that sits in front of the Manager, cutting costs and preserving strict safety:

- **FAST:** Greetings, short factual questions, and simple rewrites. Bypasses the Manager entirely for a direct, cheap response.
- **PLAN:** The default bucket. Invokes the Manager to create a JSON execution plan, which is then evaluated by the TAS Judge.
- **DEEP:** Explicitly requests research, deep-dives, or complex planning. Triggers the full Tri-Agent Synthesis (TAS) cognitive loop.

Proof-Backed Verb Allowlist (Deterministic Learning)

The Router "learns" without machine learning. Safe read-only actions (like read, list, inspect) are stored in a SHA-256 verified JSON proof in the Librarian. When Jack successfully executes a novel read command without friction, it appends the verb to the allowlist with a proof hash. The next time, the Router fast-paths it. This is evidence-based routing.

Entropy-Based Escalation

If the FAST path returns a response with high uncertainty (entropy > 3.0 bits), the Router deterministically catches the misclassification, stores a proof in the Librarian, and automatically re-routes the prompt to the PLAN path.

Manual DEEP Override

Users can demand maximum cognitive rigor by prefixing any prompt with deep (e.g., "deep analyze this repository"). The Chassis uses a deterministic regex to strip the prefix and force the DEEP intent, bypassing the automatic classifier entirely while leaving a manual_deep_override cryptographic proof in the ledger.

State-Change Invariant Routing (The Expanded FAST Path)

Early architectures relied on a mathematically naive token-length limit to route intents, forcing long but harmless queries into expensive planning loops. The modern Chassis replaces this with a State-Change Invariance Filter. The Router statically parses the user's prompt using RE2 regular expressions to hunt for system-mutation signatures (e.g., file writes, unallowlisted shell execution, or network access). If zero system-mutation intent is detected-such as a request to translate a massive document, summarize logs, or answer a factual query-the Router fast-paths the prompt directly to the highly efficient tactical model, regardless of token length. Crucially, this FAST path

deterministically intercepts @file references, injecting visual and textual evidence into the Hot Context before execution. This allows power users to harness native reasoning at maximum speed without paying the token or latency tax of the massive planning engine.

V. HIGH IRQ: THE OMEGA LINKAGE & CANARY TOKENS

High IRQ is Jack's streaming safety boundary. Its purpose is to make live token governance deterministic under three pressures: split-token credential leakage, asynchronous stream lifecycle cleanup, and retry-layer masking.

The Cognitive Mirror (Entropy Gate)

Before a stream is even processed, Jack calculates the mean entropy of the model's logprobs. If the entropy exceeds 4.0 bits, the model is mathematically hallucinating or confused. Jack triggers an immediate SovereignInterrupt (violation_type="COGNITIVE_MIRROR_ENTROPY_FAILURE"). The Engine is not allowed to guess when generating actionable output.

The Disruptor & Chunked Sliding-Window Audit

The IRQ maintains a user-configurable sliding window (defaulting to 256 characters). New text enters the window and is held. The window is audited for hard invariant violations. Power users operating in zero-trust environments may expand this window via config.yaml to quarantine massive cryptographic payloads at the cost of initial streaming latency. Only text that has aged out of the back of the window is yielded downstream.

For generated files, Jack introduces the Chunked Sliding-Window DLP Audit. Instead of loading a massive generated file into RAM (which causes an OOM crash), the StepExecutor scans the file in strict 8KB chunks. Crucially, it maintains a 256-byte overlap buffer between chunks. This guarantees that a credential split perfectly across an 8KB boundary (e.g., AWS keys) is caught and the file is shredded.

The 3-Tier Canary Token Tripwire

The High IRQ sliding window acts as the deterministic execution gate for Jack's prompt canary defenses. Rather than trusting the probabilistic Engine to enforce its own safety boundaries, the Chassis treats prompt and secret leaks as detectable network-level events. At boot, the Chassis compiles three distinct tiers of Canary Tokens into active, instance-level regular expression patterns in the ContractValidator:

- **Tier A: Auto-Vault Canaries (Implicit Secrets):** The Chassis temporarily unlocks Jack Vault at bootstrap, reads the decrypted secret values (such as API keys), registers them as exact-match canary patterns, and immediately locks the Vault to clear plain-text RAM footprints.
- **Tier B: Custom Canaries (Explicit Strings):** Developers can define custom string ciphers, project codenames, or honey-tokens in jack/canaries.yaml. To prevent the Engine from discovering these boundaries, the FileInterceptor enforces a strict Dead Zone, blocking any read or scan of jack/ and raising a DEAD_ZONE_VIOLATION.

- **Tier C: Dynamic Session Canaries (Dynamic Tripwires):** Every session generates a random UUID prefix (SYS_AUTH_[ID]) that is appended to the system prompts of Engine-facing pillars (Manager, Muscle, Judge) after prompt registry hashing checks. The token is registered as a dynamic match in the ContractValidator.

If a prompt injection tricks the model into leaking system instructions or secrets, the StreamingIRQ detects the token pattern inside its 256-character sliding window. The stream is severed instantly, raising SovereignInterrupt (violation_type='DLP_VIOLATION_CANARY_TOKEN_TRIPPED'). The Engine is never informed which string triggered the block, preventing iterative jailbreak learning.

The Reasoning Airlock (Dual-Stream IRQ Topology)

Modern probabilistic Engines increasingly utilize "Thinking" or "Reasoning" architectures, streaming an internal monologue (often via out-of-band reasoning_content deltas or <think> blocks) prior to generating their final response. In conventional agent frameworks, this reasoning stream is treated as benign diagnostic output and passed to the terminal unmonitored. Jack recognizes this as a critical exfiltration vector. If the Chassis only polices the final content stream, a compromised Engine could easily leak API keys, expose system prompts, or trip Canary Tokens entirely within its internal monologue, bypassing the safety boundary.

To neutralize this threat, Jack implements a Dual-Stream IRQ Topology. The Omega Link instantiates two entirely independent StreamingIRQ sliding windows—one for the final content stream, and a secondary "Reasoning Airlock" strictly for the reasoning_content stream. As the Engine "thinks," every token of its internal monologue is pushed through the 256-character Reasoning Airlock. It is subjected to the exact same real-time ContractValidator, DLP, and Canary Token checks as the final output. The Chassis does not distinguish between a thought and a statement. If the Engine attempts to leak a Vault secret or trips a dynamic session canary while thinking, the Reasoning Airlock instantly severs the connection and raises a SovereignInterrupt. The Engine is violently terminated before the thought can fully materialize on the host system. Post-stream, the Chassis performs a final, static DLP audit on the fully accumulated reasoning block to defeat advanced sliding-window truncation evasion. Crucially, most models can operate with native thinking turned off to significantly improve direct instruction following and JSON/XML schema adherence on tactical tasks, relying instead on the Chassis to coordinate macro-reasoning.

In-Memory Vault Defense (Secrets-at-Rest)

While the Chassis handles last-mile lookup of credentials, decrypted keys present an ephemeral memory-scraping vector. Jack mitigates this by storing decrypted Vault secrets strictly as mutable bytearray buffers in RAM. The instant a request boundary closes, these buffers are physically overwritten with null bytes, preventing plaintext credentials from lingering in memory footprints where they could be harvested via core dumps or debugging hooks.

V.A THE MATHEMATICAL DETAILS OF REAL-TIME HALLUCINATION DETECTION

Jack re-frames the problem: Hallucination is a measurable state of elevated statistical uncertainty in the Engine's local prediction space. When generating actionable code, parameters, or plans, any drift from factual accuracy correlates directly with a flatter, high-entropy token probability distribution.

1. Truncated Shannon Entropy Calculation:

For each generated token, Jack's CognitiveMirror intercepts the top-k logprobs (where $k \leq 20$ represents the API's top logprob window). Let L_i be the logprob of token i . The raw probability is $p_i = e^{L_i}$. Because the API only returns the top-k tokens, we normalize the raw probabilities over the returned set to find the localized mass $P_{\text{ext_sum}}$:

$$P_{\text{ext_sum}} = \sum_{j=1}^k p_j$$

$$P_{i,\text{normalized}} = p_i / P_{\text{ext_sum}}$$

The normalized Shannon entropy $H(X)$ is computed as:

$$H(X) = -\sum_{i=1}^k P_{i,\text{normalized}} \log_2(P_{i,\text{normalized}})$$

2. The Grace Period and Rolling Audit:

Jack applies a grace window ($\text{COGNITIVE_MIRROR_GRACE_TOKENS} = 5$) during stream boot. Once past this window, the system maintains a running weighted average of the entropy of the generated token stream:

$$H_{\text{avg}} = \left(\sum_{t=1}^N H_t \right) / N$$

If the running average H_{avg} crosses the mathematical threshold of 4.0 bits, the Engine is determined to have left its factual, calibrated reasoning regime and entered a guessing state. The connection is instantly severed with a SovereignInterrupt.

VI. MATHEMATICAL SOVEREIGNTY: THE ZERO-NETWORK EXECUTION RING

The industry standard is to allow an Engine to execute code directly via an eval() or exec() prompt-an approach that opens the door to Remote Code Execution (RCE) via adversarial payload obfuscation. Jack unequivocally rejects this. Jack's approach to mathematical analysis and scientific computation is rooted in strict host-isolation, deploying a Zero-Network Docker Sandbox that grants the Engine massive computational leverage without compromising the Chassis.

The Sandbox Airlock (Fail-Closed Execution)

When the Muscle drafts code to solve a complex computational problem, the execution is offloaded entirely from the host. Jack orchestrates an ephemeral python:3.11-slim Docker container explicitly constrained by mem limit="512m" and cpu_quota=50000. Most critically, the container operates with network_mode="none". An obfuscated reverse-shell payload generated by a hallucinating Engine screams into the void, blocked from accessing any host sockets. If an Engine hallucination causes an infinite loop or excessive recursion, the container is violently terminated by the host kernel daemon.

Z3 SMT Solvers & Bounded Verification

To handle complex logic and rigorous mathematical proofs, Jack automatically injects a hardened mathematical header into the Sandbox, pre-importing tools like numpy, sympy, and z3. This transforms Jack from a mere code-executor into a bounded formal verification engine. The Manager can propose a logical theorem or cryptographic constraint, and the Muscle will formulate it into a Z3 Satisfaction Modulo Theories (SMT) script. The deterministic Z3 backend solves the logic completely offline, returning a guaranteed sat or unsat state to the Hot Context.

Immutable Data Volumes (The Offline Oracle)

Because Jack's Sandbox operates without network access, the Engine cannot ping external APIs (like Wolfram Alpha) for physical constants or reference data mid-calculation. To bridge this gap securely, Jack employs Immutable Data Volumes. Read-only datasets (like Parquet or SQLite files) are dynamically bound to the Sandbox volume (/datasets: ro). The Engine is granted access to gigabytes of empirical reference data with absolute zero risk of data exfiltration or Server-Side Request Forgery (SSRF) attacks.

Unification of Untrusted Retrieval and Computation Boundaries

Jack strictly segregates untrusted computation from untrusted web retrieval. While the Zero-Network Sandbox is kept entirely offline to prevent remote code execution exfiltration, web retrieval tools (the WebNavigator browser) must necessarily access the network. To prevent the Engine from manipulating the browser into executing SSRF or DNS Rebinding attacks against local subnets, private ranges, or cloud metadata endpoints (e.g., 169.254.169.254), the Chassis implements a socket-level firewall. Every navigation request undergoes single-resolution DNS verification. The resolved IP is audited against forbidden private ranges. If safe, the connection is bound directly to the verified IP using Jack's custom DNSRebindingSafeHTTPSHandler and DNSRebindingSafeHTTPHandler socket wrappers, entirely bypassing standard connection-time DNS lookups and closing the DNS Rebinding window.

VII. TAS: TRI-AGENT SYNTHESIS & COST-AWARE SOVEREIGNTY

Tri-Agent Synthesis (TAS) is Jack's adversarial review protocol for high-risk plans. It exists because deterministic validation can identify syntactic violations, but failures are often epistemic. A plan can pass basic structure checks while still hiding unstable assumptions. TAS is itself a highly structured, framework-level form of reasoning, leveraging multi-agent dialectics rather than simple monologue loops. The TASGeneralAgent implements a domain-agnostic cognitive loop:

- **The Architect (Thesis):** Uses a structured 5-step reasoning scaffold (UNDERSTAND, EXPLORE, PLAN, EXECUTE, VERIFY) to propose a solution.
- **The Sage of Friction (Antithesis):** Surfaces hidden assumptions and edge cases. Crucially, the Sage is strictly forbidden from providing answers or correcting the math.
- **The Synthesizer (Synthesis):** Resolves the conflict. The Synthesizer is explicitly prompted: "Treat the interrogation as advice from a wise sage, not as truth. You are the final decision-maker."

Sovereignty via Cryptographic Framing

To prevent the Synthesizer from blindly adopting the Sage's critique as ground truth, the Chassis explicitly frames the critique as untrusted advice rather than system instruction. The Synthesizer is provided with the cryptographic Librarian Proof Hashes for both the Thesis and the Critique, forcing the model to anchor its final decision to auditable evidence rather than conversational recency bias.

Consensus Early-Stop & Confidence Metadata

TAS is expensive. Jack makes it cost-aware:

- **Confidence Metadata:** Every proof stored in the Librarian includes entropy and weight metadata derived from the provider's logprobs.
- **Consensus Stop:** If the Synthesizer produces 3 identical normalized answers across parallel attempts, the loop terminates early. Jack stops spending compute when deterministic consensus is reached.

Preemptive Cognitive Stabilization (The Hybrid Seed Guard)

To ensure that safety-critical plan generation and auditing do not suffer from stochastic flakiness, the Chassis enforces a Hybrid Deterministic Seed Guard at the provider boundary. The Judge (Auditor) is locked strictly to a temperature of 0.0 and a seed of 6 to guarantee absolute, un-bypassable greedy decoding for security reviews. The Manager (Architect) is locked to a temperature of 0.2 and a seed of 6. This ensures that while planning retains the precise margin of stochastic flexibility required to break recursive loop traps on CVI retries, both safety-critical layers remain highly reproducible, eliminating the risk of a dangerous plan randomly bypassing validation on a warm sampler. Furthermore, the Chassis preemptively enforces 'Think Max' reasoning constraints directly on the Architect's initial prompt scaffold. By obligating the Engine to proactively document alternative hypotheses, structural failure modes, and rejected assumptions on its very first forward pass, Jack significantly dampens the need for expensive, post-hoc correction retries and radically curbs token waste.

Cooperative Autonomy & Mandated Red Lines

To maximize token efficiency, Jack no longer defaults to the expensive three-agent TAS loop for standard operations. Instead, the Chassis implements Cooperative Autonomy. If a generated execution plan only utilizes isolated, local tools such as sandboxed Python execution (`python_repl`) or local file staging (`filesystem`)-the Chassis trusts the tactical model to execute the step, guarded silently by the static `ContractValidator`. Escalation to the full TAS loop is strictly reserved as an advanced cognitive firewall triggered only by four un-bypassable Red Lines:

1. Explicit User Demand
2. Unallowlisted Host-Level Mutation
3. Cross-Boundary Network Interaction
4. Cognitive Mirror Friction (Entropy: 3.0 bits)

Epistemic Synthesizer Hardening

When the TAS loop is triggered, the Synthesizer does not act as a passive moderator. To prevent the "Compromise Trap," the Synthesizer is structurally forced to operate with absolute epistemic independence. It is instructed to treat both the Architect's Thesis and the Sage's Antithesis as untrusted, adversarial inputs. It is strictly forbidden from seeking a middle-ground compromise. Instead, it must act as an uncompromising fact-checker, explicitly justifying its final validation or invalidation by cross-referencing the arguments against empirical Sandbox compile results and Librarian proof hashes.

VII.A NATIVE "THINKING-OFF" DECOUPLING: MACRO VS. MICRO REASONING

One of Jack's most profound structural safety optimizations is the strict logical boundary between macro-agentic reasoning (coordinated by the deterministic Chassis) and micro-native thinking (run by the underlying probabilistic model). In typical agent frameworks, developers enable a model's native "thinking" or "reasoning" mode across all steps, allowing the LLM to output extensive, unconstrained, out-of-band token monologues before producing its final payload. Jack completely re-evaluates this architecture. The Tri-Agent Synthesis (TAS) protocol is itself a highly structured, framework-level macro-reasoning loop. Because the Chassis orchestrates and enforces the rigorous dialectical interplay between the Architect's Thesis and the Sage's Antithesis, the individual model workers do not require native, micro-level thinking during intermediate generation stages. Jack enforces a strict "Thinking-Off" / "Reasoning-Off" discipline across the tactical plane:

- **Thesis (Architect) and Antithesis (Sage):** These roles execute with native model thinking explicitly disabled (`preserve_thinking=False`). Disabling the model's internal monologues forces them to satisfy their prompt mandates directly, prevents conversational bloat, dramatically increases token throughput, and respects local hardware constraints.
- **Manager (Planning) and Muscle (Execution):** Both pillars are configured to run with native thinking off. This ensures the Manager generates clean, parser-ready XML or JSON execution plans that are not contaminated by monologue leakage, and that the Muscle outputs directly actionable code or terminal payloads.
- **Synthesis Phase:** Only the culmination of a PLAN or DEEP run-the Master of Synthesis-benefits from having native thinking enabled (`preserve_thinking=True`). As the ultimate epistemic arbiter, the Synthesis engine is permitted to use a native monologue within its designated `<deliberation>` block, giving it the exact token-level latitude required to adjudicate the conflict and cross-reference adversarial critiques against the authoritative physical evidence produced by the local Sandbox.

By decoupling thinking at the Chassis layer, Jack prevents intermediate models from getting trapped in recursive self-referential monologue loops. The macro-level TAS loop is robust enough to carry the cognitive weight of the system, leaving individual tactical tasks to execute at maximum deterministic efficiency.

VIII. THE GHOST LEDGER: CRASH-SAFE MEMORY & QUARANTINE

Jack is local-first because agent authority is most dangerous when state boundaries are invisible. The local Chassis owns state and treats every durable transition as an auditable event.

Crash-Safe Atomic Commits

The Quarantine lifecycle is: Quarantine → Audit → Commit → Zeroization. In the Jack Beta, this is crash-safe. Files are written to jacktmp and swapped via `os.replace()` inside a try/finally block that guarantees zeroization (`strict_discard()`) even if the power is cut or an `IOError` occurs mid-copy. There are no torn writes. There is no forensic residue. Critically, the Ghost Ledger is distinct from the Sandbox's ephemeral map-reduce state. When the Muscle chunks complex calculations, intermediate states (numpy arrays or JSON) are stored safely within the Sandbox's /workspace volume and vanish when the container is purged. The Ghost Ledger is strictly reserved for finalizing durable commits to the host filesystem, ensuring temporary calculations never pollute the airlock.

Librarian Proof Enforcement

The Librarian is not a passive bucket; it is an active verifier. When Eyes proposes a Markdown Proof, the Librarian:

1. Enforces the exact 5-key schema (source, method, sha256, bytes_extracted, markdown).
2. Recomputes the SHA-256 hash on the markdown. If it does not match the proposed hash, the proof is rejected.
3. Verifies the byte count.
4. Rejects any source path containing absolute directories or traversals-leveraging string-level inspection for absolute security across all operating systems.

Sovereign Trajectory Resumption

To maximize recovery efficiency without compromising safety, the quarantine lifecycle is upgraded with Sovereign Trajectory Resumption. During active CVI recovery loops, the StepExecutor maintains a transaction-safe jack/trajectory_[session_id].json log. Successful steps are serialized as SHA-256 signatures. On retry, the executor fast-forwards by replaying cached outputs and automatically restoring any base64-encoded staged files back into the active Ghost Ledger quarantine. This guarantees that non-idempotent real-world actions are executed exactly once. Once the plan is successfully audited and committed, the temporary trajectory cache is cleanly purged from disk.

VIII.A THE POWER OF EXPANDED KNOWLEDGE (THE SOVEREIGN POLICY ENGINE & XML ANCHORING MECHANICS)

Under complex operations, even highly capable models suffer from Attention Dilution. As more context accumulates, the model's ability to maintain focus on strict architectural constraints degrades. Jack resolves this by transforming the expanded knowledge layer from a passive data repository into the user's direct Sovereign Policy Engine and Command Interface for the Jack Chassis. Rather than presenting Retrieval-Augmented Generation (RAG) as a static database for Jack to look up facts, this framework operates as a live, declarative constraint gateway where the user exercises absolute authority over the Probabilistic Engine. By appending structured rules to this layer, the user holds the power to dynamically re-program Jack's runtime boundaries and write system-level laws that the deterministic Chassis must enforce all completely at runtime, without touching a single line of the

underlying Python codebase. This allows immediate custom-tailoring of Jack's coding style, security tolerances, and domain-specific operational rules.

Instead of querying full source text, the RAG loop retrieves these user-authored structured `<pattern>` schemas using Token-Compressed XML Knowledge Anchoring in the Librarian:

```
<pattern domain="reasoning" name="Adversarial Counterexample First">
<why>Searching for the smallest counterexample before proof prevents elegant but false invarian
<how>State the claim in quantifier form, then attempt to violate it with the lowest-dimensional
<pitfall>Overfitting to a single counterexample can hide the true failure surface in a larger p
</pattern>
```

The Mechanics of User-Driven Attention Pruning

Standard documentation only specifies positive intent (how things should work), which models can easily drift away from. The XML anchoring schema splits information into two opposing, high-contrast axes to drive active attention pruning:

- **The Constructive Axis (`<how>`):** Explicit code-level execution guidelines, providing the Engine with the exact architectural blueprints and coding structures the user wants implemented.
- **The Restrictive Axis (`<pitfall>`):** Explicit negative constraints highlighting specific failure modes, edge cases, or security vulnerabilities.

Negative constraints act as highly effective mathematical pruners during auto-regressive sampling. When the user inputs high-level information bit by bit through this gateway, they are not merely giving Jack casual tips; they are actively guiding the Engine's token sampling. Because the user's policy inputs directly alter the active HotContext, defining custom pitfalls mathematically forces the local model's attention heads to evaluate and prune dangerous generational paths before the Muscle compiles or synthesizes a single token. This micro-specification anchoring allows compact local models (such as Gemma 4 12B) to maintain strict architectural compliance without bloating the context window or diluting their core task focus.

Complete Reference Example of a User-Defined Sovereign Policy

The following complete, copy-pasteable configuration demonstrates how easily a user can define a custom security or execution rule that the Jack Chassis will strictly enforce at the runtime boundary:

```
<pattern domain="security" name="Sovereign Vault Isolation">
<why>To guarantee that sensitive API keys, credentials, and passwords never leak into the model
<how>when drafting execution steps or scripts that require external service authentication, you
<pitfall>Placing raw, un-redacted API keys, bearer tokens, or password strings directly into th
</pattern>
```

VIII.E MULTIMODAL ASSET QUARANTINE AUDITS (FREQUENCY & VISUAL DLP)

When Jack's probabilistic Engine generates multimodal content-such as synthesized speech (Audio Studio, Pillar 7) or high-fidelity images (Visual Studio, Pillar 6)-these binary payloads present novel, out-of-band threat vectors, including data exfiltration via image-based steganography or malicious ultrasonic commands embedded in audio files. Jack compresses these threats within the isolated GhostLedger quarantine before any asset is permitted to commit to the host filesystem:

- **Deterministic Audio Frequency Auditing (The Ultrasonic Gate):** Before any generated wav audio asset is committed, the Chassis performs an offline Fast Fourier Transform (FFT) spectral analysis of the raw PCM signal. If the analysis detects high-energy ultrasonic frequencies (exceeding 20,000 Hz) that represent more than 5% of the audible range's peak energy, the Chassis identifies the signal as an adversarial payload. The Chassis immediately raises a SovereignInterrupt (violation_type="AUDIO_STEGANOGRAPHY_DETECTED") and shreds the steganographic active quarantine buffer.
- **Multimodal Visual DLP Auditing:** For generated images (png, jpg, jpeg), the StepExecutor base64-encodes the quarantined binary and routes it directly to our multimodal perception Engine (Eyes, Pillar 4) inside a highly constrained, closed-loop system prompt: "Identify any text in this image that looks like an API key, password, or credential. Reply ONLY with 'CLEAN' or 'LEAK'." If the perception Engine returns LEAK, or if the subsequent Master Auditor (Judge, Pillar 2) flags the generated visual asset as unsafe, the runtime memory buffer is physically zeroized with NUL bytes, the transaction is rolled back, and the run is aborted to prevent out-of-band credential exfiltration.

Active Memory Management & Defragmentation Mechanics

- **The 75% Watermark Trigger:** The StepExecutor continuously tracks token utilization within the active HotContext window. The moment the active buffer reaches 75% capacity, the Chassis intercepts the execution loop and freezes further token streaming.
- **The 50% Librarian Compaction Pass:** The frozen conversational history is offloaded to the Librarian pillar. The Librarian runs a high-density compaction pass that strips conversational fluff, discards redundant intermediate execution states, and condenses the raw log history by a minimum of 50%.
- **Markdown Checkpoint Serialization:** The compressed history is serialized into a system-trusted Markdown summary checkpoint. The Librarian computes a SHA-256 provenance hash over this text block, signing it as a verified historical baseline.
- **HotContext Re-Seeding:** The active memory buffer is physically zeroized, immunized by the Type-Coercion Guard, and re-seeded using only the cryptographically signed Markdown checkpoint. This drops the active token count well below the danger threshold, restoring physical host RAM without losing high-value semantic context.

IX. THE TELEMETRY KILL SWITCH

Open-source users run Jack on proprietary local repositories. Jack includes a global `telemetry_enabled` flag. When disabled, Jack runs completely dark. If a Sovereign Interrupt or DLP violation occurs, the Chassis deterministically redacts the error message to "Redacted local error." and clears all metadata payloads before emitting the event. Sensitive local state never leaks to external callbacks.

X. THE GEMMA 4 & GEMINI HYBRID TOPOLOGY: REFERENCE IMPLEMENTATION

The Beta Gemma 4 Edition demonstrates how Jack's Chassis can coordinate multiple local and cloud-based model workers without merging their responsibilities or causing cognitive role collapse. This topology implements Hierarchical Distributed Cognition, cleanly dividing operations across Strategic, Adjudicative, and Tactical planes to prevent configuration ambiguity:

- **Gemma 4 26B (Local Macro-Agentic Planning & Local Security Plane):** Assigned exclusively to local, high-context planning (Manager). It constructs the multi-step JSON execution plans that drive the task trajectory, validating structural bounds locally before any execution can occur.
- **Gemma 4 12B (Local Micro-Agentic Execution / Local Tactical Plane):** Natively powers the Muscle, Eyes, Visual Studio, Audio Studio, and the Sage of Friction (Antithesis). By utilizing its native multimodal capabilities, the Gemma 4 12B handles rapid, high-fidelity asset generation, seamless local tool execution, and drives visual, evidence-backed ingestion and interrogation.
- **Gemini-3.1-Flash-Lite (Cloud-Delegated Adjudication & Fast Auditing Plane):** Mapped to the Judge and the Master of Synthesis (`tas_synthesis`). Because the final Master of Synthesis must evaluate both local proposals (Thesis) and critiques (Antithesis) under strict JSON schema constraints, delegating this role to Gemini-3.1-Flash-Lite drastically minimizes local hardware overhead while ensuring blazingly fast turnarounds. This cloud execution is rendered entirely safe through Jack's host-level containment gates which prevent local credentials, directories, or sandboxed execution context from leaking out.
- **Embedding Model (Librarian-Embed / Librarian-Rerank):** Dedicated strictly to the Librarian for high-speed vector indexing and RAG retrieval across the Shadow Ledger.

This hybrid layout ensures that a developer setting up a `config.yaml` maps the highly efficient local Gemma 4 12B to execution-heavy and perception-heavy roles, delegates the high-throughput auditing and synthesis tasks to Gemini-3.1-Flash-Lite, and reserves the local Gemma 4 26B footprint purely for planning. The Chassis remains supreme by mechanism, refusing to allow a single model worker to slip into structural self-auditing.

XI. WHY JACK IS DIFFERENT FROM OTHER AGENTS

Jack differs from conventional agents in its refusal to merge competence with control. Many agent frameworks improve the model, expand the tool list, add memory, and hope that a stronger prompt produces safer behavior. Jack assumes that stronger models can still fail, and that safer behavior must be enforced outside the model.

A NORMAL AGENT SAYS:

"The model decided to run this."

JACK ASKS:

"Did the deterministic Chassis authorize this state transition?"

A NORMAL AGENT SAYS:

"The model read the file."

JACK ASKS:

"Was the file converted into Markdown Proof with a cryptographically verified digest?"

A NORMAL AGENT SAYS:

"The model generated a useful artifact."

JACK ASKS:

"Did the artifact survive the chunked sliding-window audit, and was the quarantine atomically zeroized?"

Jack's most important feature is not that it can call an LLM. Jack's most important feature is that it can say no in ways the model cannot override.

Jack as the Ultimate Harness for DiffusionGemma

With Google DeepMind's release of DiffusionGemma (diffusiongemma-26B-A4B-it), local-first inference is transitioning from left-to-right causal autoregressive generation to parallel, iterative block-based denoising (Discrete Text Diffusion or dLLMs) [14]. While this architecture unlocks blistering throughput—refining an entire 256-token canvas in parallel at speeds exceeding 150 tokens/second locally—it introduces a critical trade-off: DiffusionGemma lacks native tool-calling capabilities and has lower relative reasoning quality than standard causal models [14]. Left unconstrained, it is highly prone to formatting drift, syntax errors, and structural collapse.

Jack is the perfect mathematical and architectural stabilization harness for DiffusionGemma:

Chassis-Side Parsing (No Native Tool-Calling Required): Jack does not rely on proprietary, model-level function-calling APIs. Instead, the Manager (Pillar I) instructs the dLLM to propose plans inside standard XML tags or raw markdown JSON. Jack's deterministic Chassis parses this text using robust regular expressions (re2-backed) and converts it into structured, validated Pydantic models.

Loose Heuristic Extraction: DiffusionGemma can sometimes output unstable or un-nested plan blocks under pressure. Jack's newly upgraded Loose Step Extraction engine gracefully extracts, repairs, and executes these loose blocks, shielding the execution loop from parser crashes.

Active Attention Pruning via Pitfall Anchoring: Custom high-contrast XML patterns retrieved from the Sovereign Policy Engine (Expanded Knowledge) are injected directly into the active HotContext [1]. The explicit, negative-space <pitfall> constraints mathematically prune the dLLM's bidirectional attention passes, forcing the iterative parallel denoising process to quickly converge on stable, safe, and validated token configurations [2].

Bounded Formal Verification: Because the dLLM is less mathematically robust, Jack never lets it calculate numeric answers. The prompt-level Proof Gate forces the dLLM to draft a formal Z3 SMT script, utilizing our offline sandbox fallback to compute guaranteed logical truths.

By pairing DiffusionGemma's hyper-fast parallel throughput with the strict, fail-closed boundaries of the deterministic Chassis, developers can finally harness the speed of discrete text diffusion with absolute host and operational safety.

XII. CONCLUSION: LEAD-LINED AUTONOMY

Jack is a lead-lined chassis for autonomous intelligence. It does not deny the power of language models. It denies their sovereignty. The Engine can be brilliant and still be untrusted. The Chassis can be simple and still be supreme. This is the essential design move: separate cognition from authority, then make every authority transition deterministic, inspectable, and interruptible. High IRQ and the Ghost Ledger strengthen that move at the most dangerous moments in an agent run: the live stream and the filesystem commit.

The moment text begins to emerge from a probabilistic Engine, conventional systems are tempted to show it, log it, route it, or act on it. Jack holds it behind a quarantine window. The validator inspects it. The Cognitive Mirror measures its entropy. The IRQ releases only safe text. On a hard violation or canary trip, Jack zeroizes and strikes. On cleanup trouble, Jack preserves the strike. On retry temptation, Jack refuses to launder the violation into another model prompt. It is not a prompt. It is not a personality. It is not a refusal style. It is deterministic software disposing of probabilistic cognition.